

```
// server.js
require('dotenv').config();
const express =
require('express');
const mongoose =
require('mongoose');
const axios = require('axios');
const cors = require('cors');
```

```
const app = express();
app.use(express.json());
app.use(cors());
```

```
// 1. CONNEXION À LA BASE DE
DONNÉES
```

```
mongoose.connect(process.env.
```

MONGO_URI)

.then(() => console.log("📁

Connecté avec succès à
MongoDB"))

.catch(err => console.error("❌

Erreur de connexion MongoDB :",
err));

// 2. MODÈLE DE DONNÉES
UTILISATEUR (SCHEMA)

const UserSchema = new
mongoose.Schema({

telegramId: { type: String,
 required: true, unique: true },

soldeFixe: { type: Number,
 default: 0.15123025 }, // Dernier

solde enregistré

 derniereMiseAJour: { type:
Date, default: Date.now }, // Date
du dernier calcul

 isPremium: { type: Boolean,
default: false },

 machines: [
 {
 idMachine: String,
 nom: String,
 puissance: Number, // En
Th/s

 dateAchat: { type: Date,
default: Date.now },

 expiration: Date // Date
exacte d'expiration, ou null si

```
    illimité  
  }  
]  
});
```

```
const User =  
mongoose.model('User',  
UserSchema);
```

```
// 3. FONCTION DE SÉCURITÉ :  
RECALCULER LE SOLDE RÉEL  
async function  
actualiserEtObtenirSoldeReel(tele  
gramId) {  
  const user = await  
User.findOne({ telegramId });
```

```
if (!user) return null;
```

```
const maintenant = new Date();  
const tempsEcouleSecondes  
= (maintenant - new  
Date(user.derniereMiseAJour)) /  
1000;
```

```
if (tempsEcouleSecondes <=  
0) return user.soldeFixe;
```

```
// Calcul de la puissance  
totale actuelle valide
```

```
let puissanceTotaleActive = 0;
```

```
// Si Premium, il a le bonus
```

permanent de 5.8 Th/s

if (user.isPremium)

puissanceTotaleActive += 5.8;

user.machines.forEach(machine

=> {

 // Vérifier si la machine est
toujours valide (pas expirée)

 if (!machine.expiration ||
new Date(machine.expiration) >
maintenant) {

 puissanceTotaleActive +=
machine.puissance;
 }

});

// Formule mathématique
identique au frontend pour les
gains accumulés

```
const gainsAccumules =  
tempsEcouleSecondes *  
(puissanceTotaleActive *  
0.0000000001);
```

// Sauvegarde du nouveau
solde en base de données

```
user.soldeFixe +=  
gainsAccumules;  
user.derniereMiseAJour =  
maintenant;  
await user.save();
```

```
    return user.soldeFixe;  
}
```

// 4. API : RÉCUPÉRER LE
PROFIL (Appelée au chargement
de la Mini App)

```
app.get('/api/user/  
profile/:telegramId', async (req,  
res) => {  
    try {  
        const { telegramId } =  
req.params;  
        let user = await  
User.findOne({ telegramId });
```



```
// Si l'utilisateur est
nouveau, on lui crée un compte
if (!user) {
    user = new User({
        telegramId,
        machines: [{ idMachine:
'antminer_s19', nom: 'Antminer
S19 Pro', puissance: 110.0 }] //
Machine de départ
    });
    await user.save();
}
```

```
// Recalculer son solde
exact avant de lui envoyer
const soldeMisAJour =
```

await

actualiserEtObtenirSoldeReel(telegramId);

```
    res.json({  
        success: true,  
        solde: soldeMisAJour,  
        isPremium:
```

```
user.isPremium,  
        machines: user.machines  
    });
```

```
    } catch (error) {  
        res.status(500).json({  
            success: false, message:  
            error.message });  
    }
```

```
});
```

```
// 5. API : RETRAIT
```

```
AUTOMATIQUE SÉCURISÉ
```

```
(Payout)
```

```
app.post('/api/user/retrait-  
automatique', async (req, res) =>  
{
```

```
    const { telegramId, operateur,  
    coordonnees, montant } =  
    req.body;
```

```
    const montantDemande =  
    parseFloat(montant);
```

```
    try {
```

```
        // Étape essentielle : On
```

calcule le vrai solde stocké en
base de données

```
    const soldeReel = await  
actualiserEtObtenirSoldeReel(tele  
gramId);
```

```
    if (!soldeReel || soldeReel <  
montantDemande) {  
        return  
res.status(400).json({ success:  
false, message: "Tentative de  
fraude ou solde insuffisant !" });  
    }
```

```
    // Préparation du transfert  
automatique avec l'agrégateur
```

```
const payloadPayout = {  
  apikey:  
process.env.API_KEY_AGGREGATE  
UR,  
  site_id:  
process.env.SITE_ID,  
  payment_method:  
opérateur.toLowerCase(),  
  amount:  
montantDemande,  
  currency: "XOF",  
  recipient_phone:  
coordonnées,  
  transaction_id: `PAYOUT-$  
{Date.now()}`,  
  description: `Retrait Titan
```

```
Miner pour ${telegramId}`  
};
```

// Décommenter l'appel ci-dessous lorsque tes accès de production seront actifs :

```
// const reponseAgregateur  
= await axios.post('https://api-payout.agregateur.com/v1/  
transfer', payloadPayout);
```

// Simulation de réussite pour tes tests de développement :

```
const reponseAgregateur = {  
data: { status: "SUCCESS" } };
```

```
    if
(reponseAgregateur.data.status
=== "SUCCESS") {
    // Déduire définitivement
l'argent de sa base de données
    const user = await
User.findOne({ telegramId });
    user.soldeFixe -=
montantDemande;
    await user.save();

    return res.json({
        success: true,
        message: `Le virement
automatique de $
```

{montantDemande} XOF a été
envoyé !`

});

} else {

return

res.status(500).json({ success:
false, message: "Rejet de la
transaction par l'opérateur
mobile." });
}

} catch (error) {

console.error("Erreur
serveur retrait :", error.message);
res.status(500).json({
success: false, message: "Erreur


```
technique lors du retrait  
automatique." });  
}  
});
```

```
const PORT = process.env.PORT  
|| 5000;  
app.listen(PORT, () =>  
console.log(`🚀 Serveur en ligne  
sur le port ${PORT}`));
```